

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 7
Searching

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

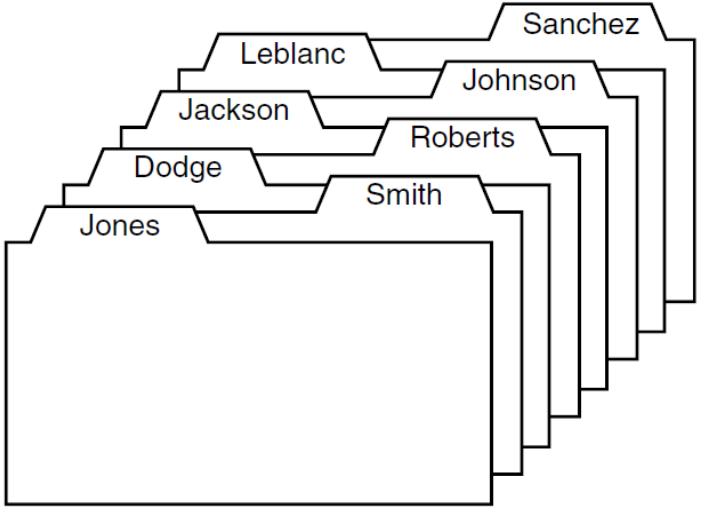
Outline

- Records and Keys
- Sequential Search
- Binary Search
- Comparison Tree for Analysis
- Applications

Introduction

- 在英汉字典中查找某个英文单词的中文解释；在新华字典中查找某个汉字的读音、含义；在对数表、平方根表中查找某个数的对数、平方根；邮递员送信件要按收件人的地址确定位置等等。可以说查找是为了得到某个信息而常常进行的工作。
- **查找是许多程序中最消耗时间的一部分。**因而，一个好的查找方法会大大提高运行速度。常用的查找结构通常分为若干类：
  - 线性结构(含线性链表结构)
  - 树形结构
  - 散列(哈希)结构

Records and Keys



- We are given a *list* of *records*, where each record is associated with one piece of information, which we shall call a *key*.
- We are given one key, called the *target*, and are asked to search the list to find

the record(s) (if any) whose key is the same as the target.

Searching

- Search may succeed if there exists one record with the same key. There may be more than one record with the same key.
- Search may fail if there is no record with the given key.

- Time complexity of the search algorithm: we count how many times one key is compared with another during a search.
- *Internal searching:* all the records are kept in high-speed memory.
- *External searching*: most of the records are kept in disk files.

Parameters for Search Functions

- Each searching function has at least two input parameters:
  - First is the *list* to be searched;
  - Second is the *target* key for which we are searching.
  - (The searching range of the list.)
- Each searching function will also have an output parameter and a returned value:
  - The returned value has type **Error_code** and indicates whether or not the search is

successful in finding an entry with the target key.

- If the search is successful, then the returned value is success, and the output parameter called position will locate the target within the list.
- If the search is unsuccessful, then the value not_present is returned, and the output parameter may have an undefined value or a value that will differ from one method to another.

Key Definition in C++

```
// Definition of a Key class:
class Key{
public:
   // Add any constructors and methods for key data.
private:
   // Add declaration of key data members here.
};
// Declare overloaded comparison operators for keys:
bool operator == (const Key &x, const Key &y);
bool operator > (const Key &x, const Key &y);
bool operator < (const Key &x, const Key &y);
```

```
bool operator >= (const Key &x, const Key &y);
bool operator <= (const Key &x, const Key &y);
bool operator != (const Key &x, const Key &y);

// Definition of a Record class:
class Record{
public:
   operator Key( ); // implicit conversion from Record to Key.
   // Add any constructors and methods for Record objects.
private:
   // Add data components.
};
```


Records and Keys in C++

- Every Record is associated to a key (of a type or **class** called Key).
- Key objects can be compared with the standard operators

  == , != , <, >, <= , >=
- There is a conversion operation to turn any Record into its associated Key.
- Record objects can be compared to each other or to a Key by first converting the Record objects to their associated keys.

- We do not assume that a **Record** has a **Key** object as a data member, although often it does. We merely assume that the compiler can turn a **Record** into its corresponding **Key**.

Sequential Search

- Algorithm: the simplest way to do search. Begin at one end of the list and scan down it until the desired key is found or the other end is reached;
- Analysis: How much work needs to be done to search a key;
- Test: empirical testing on sample data.

To analyze the behavior of an algorithm that makes comparisons of keys, we shall use the count of these key comparisons as our measure of the work done.

Sequential Search and Analysis

```
Error_code sequential_search(const List<Record> &the_list,
   const Key &target, int &position)
{
   int s = the_list.size( );
   for (position = 0; position < s; position++) {
       Record data;
```

```
            the_list.retrieve(position, data);
            if (data == target) return success;
        }
        return not_present;
    }
```

- The number of comparisons of keys done in sequential search of a list of length *n* is
  - Unsuccessful search: *n comparisons.*
  - Successful search, best case: 1 comparison.
  - Successful search, worst case: *n* comparisons.
  - Successful search, average case: (*n+1)/2* comparisons.

Testing

- Set up sample data, run the functions, and check
  - if the search function works and

- compare the results with those of the theoretical analysis.

- Two numbers worth calculating:
  - The number of key comparisons done over many searches;
  - The mount of CPU time required.

- Use the function **clock()** to provide CPU timing information.

Test Data for Searching

- Most later searching methods require the data to be ordered, so use a list with integer keys in increasing order.

- To test both successful and unsuccessful searches, insert only keys containing odd integers into the list.
- If *n* denotes the number of entries in the list, then the targets for successful searches will be 1, *3, 5, ... , 2n − 1.*
- For unsuccessful searches, the targets will be 0*, 2, 4, 6, ... , 2n.*
- In this way we test all possible failures, including targets less than the smallest key in the list, between each pair, and greater than the largest.

Sentinel

- At each iteration, sequential search checks two inequalities, one a comparison

of keys to see if the target has been found, and the other a comparison of indices to see if the end of the list has been reached.

- A good way to speed up the algorithm by eliminating the second comparison is to make sure that eventually key target will be found, by increasing the size of the list and inserting an extra item at the end with key target. Such an item placed in a list to ensure that a process terminates is called a ***sentinel***. **（哨兵，监视哨）**
- When the loop terminates, the search will have been successful if target was found before the last item in the list and unsuccessful if the final sentinel item was the one found.

Sequential search

- The method is simple;
- Efficient for short list;
- The list don’t have to be ordered;
- Disaster for long list to be searched;

There are far more efficient methods for ordered lists!

- The method requires the list is ordered;
- The method: first compare the key with the middle record in the list, then decide to continue search on the first half or the second half using the same method.
- The method reduces the number of comparisons dramatically.

DEFINITION An ***ordered list*** is a list in which each entry contains a key, such that the keys are in order. That is, if entry i comes before entry j in the list, then the key of entry i is less than or equal to the key of entry j.

Ordered Lists

- All List operations except insert and replace apply without modification to an ordered list.
- Methods **insert** and **replace** must fail when they would otherwise disturb the order of a list.
- We implement an ordered list as a class derived from a *contiguous* List. In this derived class, we shall override the methods insert and replace with new implementations.

```
class Ordered_list: public List<Record>{
public:
    Ordered_list( );
    Error_code insert(const Record &data);
    Error_code insert(int position, const Record &data);
    Error_code replace(int position, const Record &data);
};
```

Overloaded Insertion

```
Error_code Ordered_list :: insert(const Record &data)
/* Post: If the Ordered list is not full, the function succeeds:
The Record data is inserted into the list, following the last
entry of the list with a strictly lesser key (or in the first
list position if no list element has a lesser key).
Else: the function fails with diagnostic Error_code overflow. */
{
    int s = size( );
    int position;
    for (position = 0; position < s; position++) {
        Record_list data;
        retrieve(position, list_data);
        if (data >= list_data) break;
    }
    return List<Record> :: insert(position, data);
}
```

找到插入位

- The scope resolution is necessary, because we have overridden the original List **insert** with a new Ordered list method.

Binary Search

- **Idea:** In searching an ordered list, first compare the target to the key in the center of the list. If it is smaller, restrict the search to the left half; otherwise restrict the search to the right half, and repeat. In this way, at each step we reduce the length of the list to be searched by half.
- **Keep** two indices, top and bottom, that will bracket the part of the list still to be searched.
- **Initialization**: Set bottom = 0; top = the_list.size( ) − 1;
- **Compare** target with the Record at the midpoint,

  mid = (bottom + top)/2;
- **Change** the appropriate index top or bottom to restrict the search to the appropriate half of the list.
- **Loop** terminates when top <= bottom, if it has not terminated earlier by finding

the target.

- **Make** progress toward termination by ensuring that the number of items remaining to be searched, top − bottom + 1, strictly decreases at each iteration

The Forgetful Version of Binary Search

- Forget the possibility that the key target might be found quickly and continue to subdivide the list until what remains has length 1.

```
Error_code recursive_binary_1(const Ordered_list &the_list,
      const Key &target, int bottom, int top, int &position)
/* Pre:   bottom and top define the range in the list to search for the target.
   Post:  If a Record in the range of locations from bottom to top in the_list has
          key equal to target, then position locates one such entry and a code
          of success is returned. Otherwise, the Error_code of not_present is
          returned and position becomes undefined.
   Uses:  recursive_binary_1 and methods of the classes List and Record.   */
```

The Forgetful Version of Binary Search

```
{ Record data;
  if (bottom < top) {          //  List has more than one entry.
    int mid = (bottom + top)/2;
    the_list.retrieve(mid, data);
    if (data < target)         //  Reduce to top half of list.
      return recursive_binary_1(the_list, target,
                                mid + 1, top, position);
    else                       //  Reduce to bottom half of list.
      return recursive_binary_1(the_list, target,
                                bottom, mid, position);
  }
  else if (top < bottom)
    return not_present;        //  List is empty.
  else {                       //  List has exactly one entry.
```

```
        position = bottom;
        the_list.retrieve(bottom, data);
        if (data ==  target) return success;
        else return not_present;
    }
}
```


Algorithm Verification

- What do we need to verify or prove that our algorithm is correct?
  - The target key can be found if it is present in the list and it is the first occurrence.
  - The algorithm terminates.
- The division of the list into sublists is described in the following diagram:



- Only entries *strictly* less than target appear in the first part of the list, but the

last part contains entries greater than or equal to target.

- When the middle part of the list is reduced to size 1, it will be guaranteed to be the *first* occurrence of the target if it appears more than once in the list.

Nonrecursive Binary Search

```
Error_code binary_search_1 (const Ordered_list &the_list,
    const Key &target, int &position)
/* Post: If a Record in the list has Key equal to target, then position
locates one such entry and a code of success is returned. Otherwise,
not present is returned and position is undefined.
Uses: Methods for classes List and Record. */
{
    Record data;
    int bottom = 0, top = the_list.size( ) - 1;
    while (bottom < top) {
        int mid = (bottom + top) / 2;
        the_list.retrieve(mid, data);
        if (data < target)
```

```
            bottom = mid + 1;
        else top = mid;
    }
    if (top < bottom) return not_present;
    else {
        position = bottom;
        the_list.retrieve(bottom, data);
        if (data == target) return success;
        else return not_present;
    }
}
```


Recognizing Equality in Binary Search

- **Method**: Check at each stage to see if the target has been found.

```
Error_code binary_search_2(const Ordered_list &the_list,
   const Key &target, int &position)
/* Post: If a Record in the list has key equal to target, then position
locates one such entry and a code of success is returned. Otherwise,
not present is returned and position is undefined.
Uses: Methods for classes Ordered_list and Record. */
{
   Record data;
```

```
    int bottom = 0, top = the_list.size( ) - 1;
    while (bottom <= top) {
        position = (bottom + top) / 2;
        the_list.retrieve(position, data);
        if (data == target) return success;
        if (data < target) bottom = position + 1;
        else top = position - 1;
    }
    return not_present;
}
```


Algorithm Verification

- The division of the list into sublists is described in the following diagram:



- The first part of the list contains only entries strictly less than target, and the last part contains only entries strictly greater than target.
- If **target** appears more than once in the list, then the search may return any instance of the target.
- If **target** does not exist in the list, then we have ordered_list[top] < target < ordered_list[bottom]

Comparison Trees

- The comparison tree of an algorithm is obtained by tracing the action of the algorithm.

- A vertex (circle) represents a comparison of keys. Inside the circle we put the

index of the key against which we are comparing the target key.

- Branches (lines) drawn down from the circle represent the possible outcomes of the comparison.
- Leaves (squares): When the algorithm terminates, we put either F (for failure) or the location where the target is found at the end of the appropriate branch.

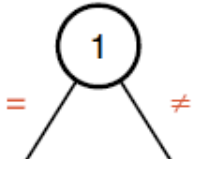
Comparison Trees

- The number of branches traversed to reach a vertex from the root is called the *level* of the vertex. Thus the root itself has level 0, the vertices immediately below it have level 1, and so on. The largest level that occurs is called the

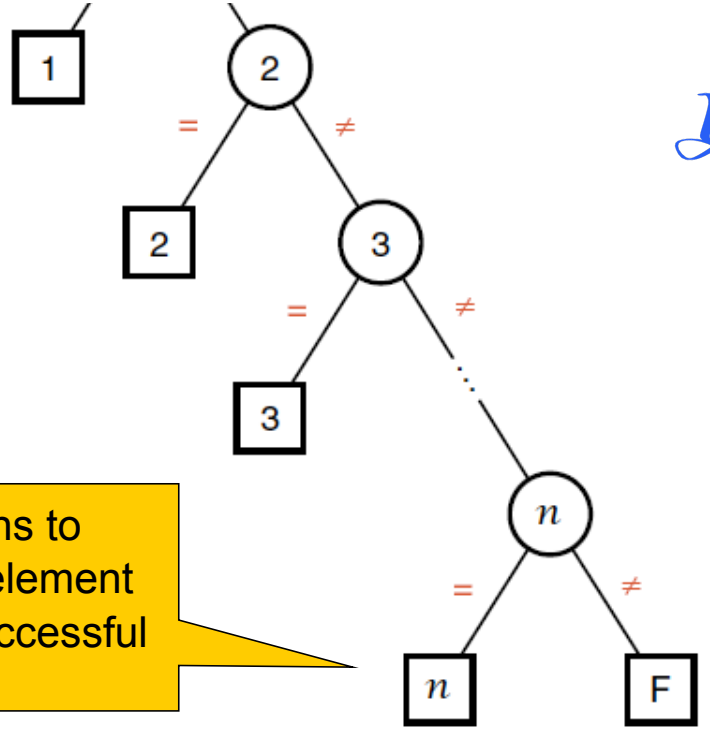
height of the tree.

- The *external path length* of a tree is the sum of the number of branches traversed in going from the root once to every leaf in the tree.
- The *internal path length* is defined to be the sum, over all vertices that are not leaves, of the number of branches from the root to the vertex.

Sample Comparison Trees

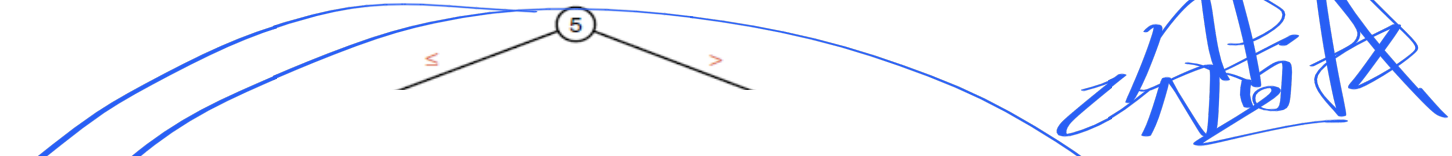


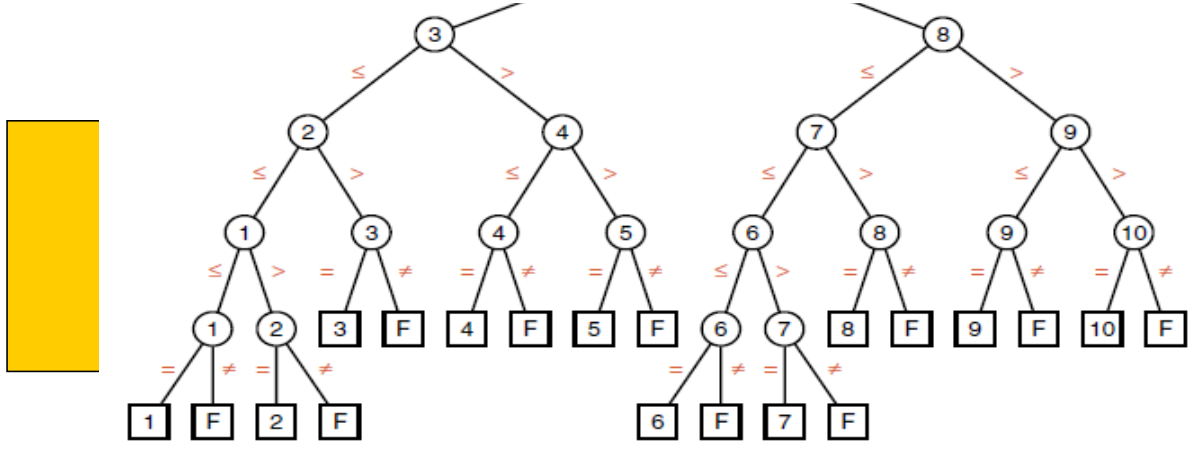
Comparison tree for sequential search on a list of n elements.



n comparisons to search *n*-th element and for unsuccessful search

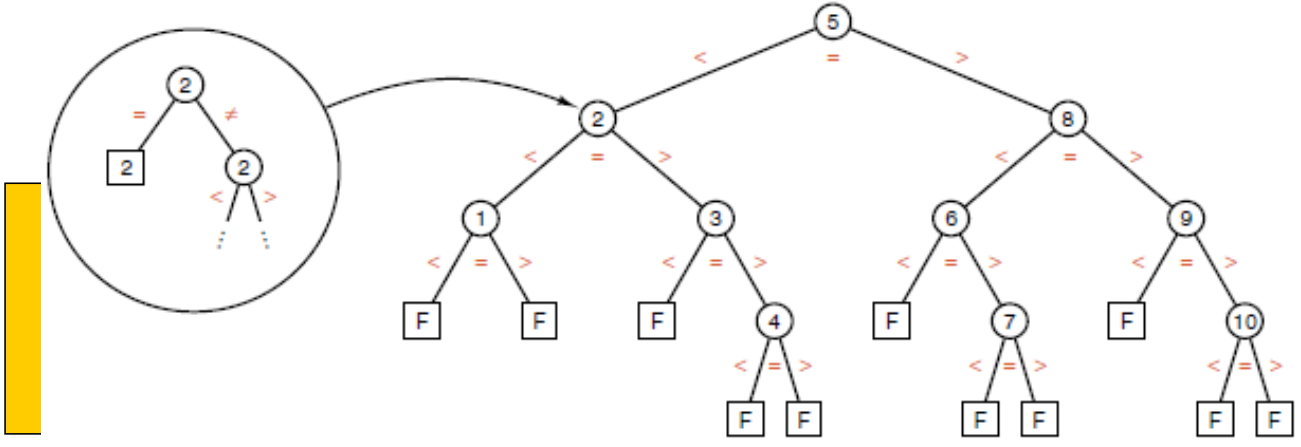
Comparison Trees: First version





- Search terminates at leaves, half are successful and half are unsuccessful;
- All leaves are on the two adjacent levels;
- Worst case: the number of comparisons is the height of the tree;
- Average case?

Comparison Trees: Second version



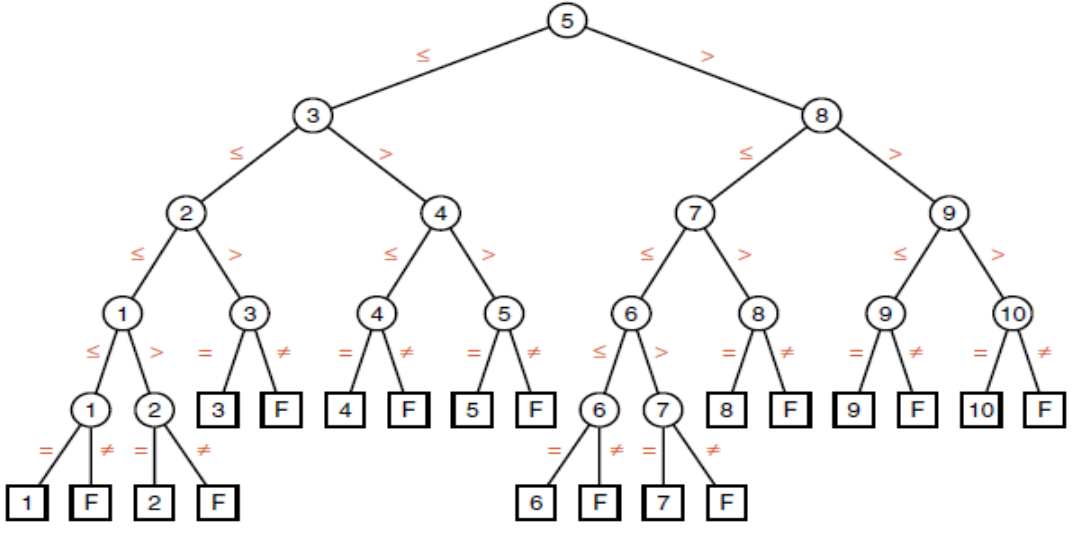
- Every vertex that is not a leaf terminates some successful search and the leaves correspond to unsuccessful searches.
- Only one comparis rtex at which the search succeeds in finding the target, c o comparisons.

Comparison Count for binary_search_1

- The total number of comparisons (for successful and unsuccessful) is the external path length of the tree, that is,

$$(4 \times 5) + (6 \times 4) + (4 \times 5) + (6 \times 4) = 88$$

- Half the leaves correspond to successful searches and half to unsuccessful searches, so the ave[illegible]nparisons for both successful and unsuccessful searches is 44/10 =



Comparison Count for binary_search_2

- The total number of successful comparisons is the *internal path length* of the tree, that is,

$$0+1+2+2+3+1+2+3+2+3=19$$

- The average number of comparisons for a successful search to be

$$2\square(19/10+1)-1=4.8$$

- For an unsuccessful search by binary_search_2, we need the *external path length* of the tree. This is

$$(5\square3)+(6\square4)=39$$

- The average number of comparisons for an unsuccessful search to be

$$(2\square39)/11\approx7.1$$

Binary Search Analysis

The number of comparisons of keys done by binary_search_1 in searching a list of n items is approximately

$$\lg n + 1$$

in the worst case and

$$\lg n$$

in the average case. The number of comparisons is essentially independent of whether the search is successful or not.

The number of comparisons done in an unsuccessful search by binary_search_2 is approximately $2\lg(n+1)$.

In a successful search of a list of n entries, binary_search_2 does approximately

$$\frac{2(n+1)}{n}\lg(n+1) - 3$$

comparisons of keys.

Binary Search Analysis

	Successful search	*Unsuccessful search*
binary_search_1	$\lg n + 1$	$\lg n + 1$
binary_search_2	$2 \lg n - 3$	$2 \lg n$

Lower Bounds for Searching

- Binary search does far fewer comparisons than sequential search.
- Question: Is there any algorithm that will in the worst case and the average case, be able to find its target using significant fewer comparisons of keys than binary search?
- The answer is NO! As long as the algorithm is based on comparisons.
- This can be proven by examine the comparison tree.

std::binary_search

- Defined in header <algorithm>

template< **class** ForwardIt, **class** T >
bool binary_search(ForwardIt first, ForwardIt last, **const** T& value);

template< class ForwardIt, class T, class Compare >
bool binary_search(ForwardIt first, ForwardIt last, **const** T& value, Compare comp);

- Checks if an element equivalent to value appears within the range [first, last).

std::lower_bound

- Defined in header <algorithm>

template< **class** ForwardIt, **class** T >
ForwardIt lower_bound(ForwardIt first, ForwardIt last, **const** T& value);

template< **class** ForwardIt, **class** T, **class** Compare >
ForwardIt lower_bound(ForwardIt first, ForwardIt last, **const** T& value, Compare comp);

- Returns an iterator pointing to the first element in the range [first, last) that

is *not less* than (i.e. greater or equal to) value.

std::upper_bound

- Defined in header <algorithm>

template< **class** ForwardIt, **class** T >
ForwardIt upper_bound(ForwardIt first, ForwardIt last, **const** T& value);

template< **class** ForwardIt, **class** T, **class** Compare >
ForwardIt upper_bound(ForwardIt first, ForwardIt last, **const** T& value, Compare comp);

- Returns an iterator pointing to the first element in the range [first, last) that is *greater* than value.

lower_bound/upper_bound Example

```
1  // lower_bound/upper_bound example
2  #include <iostream>     // std::cout
3  #include <algorithm>    // std::lower_bound, std::upper_bound, std::sort
4  #include <vector>       // std::vector
5
6  int main () {
7    int myints[] = {10,20,30,30,20,10,10,20};
8    std::vector<int> v(myints,myints+8);           // 10 20 30 30 20 10 10 20
9
10   std::sort (v.begin(), v.end());                // 10 10 10 20 20 20 30 30
11
12   std::vector<int>::iterator low,up;
```

```
std::vector<int>::iterator low,up;
low=std::lower_bound (v.begin(), v.end(), 20); //          ^
up= std::upper_bound (v.begin(), v.end(), 20); //                ^

std::cout << "lower_bound at position " << (low- v.begin()) << '\n';
std::cout << "upper_bound at position " << (up - v.begin()) << '\n';

return 0;
}
```


最大值最小化

- Problem: 把一个包含n个正整数的序列划分成m个连续的子序列（每个正整数恰好属于一个序列）。设第i个序列的各数之和为S(i)，如何让所有S(i)的最大值尽量小?
- 例如序列1 2 3 2 5 4，划分成3个序列的最优方案为1 2 3 | 2 5 | 4，其中S(1)=6, S(2)=7, S(3)=4，最大值为7；如果划分成1 2 | 3 2 | 5 4，则最大值为9，不如刚

才的好。

- n≤106，所有数之和不超过109。

最大值最小化

- 考虑一个新的问题P(x)：能否把输入序列划分成m个连续的子序列，使得所有S(i)均不超过x? (P(x)不难计算：每次尽量往右划分即可)。
- 接下来随便猜一个x0，如果P(x0)为假，那么答案比x0大；如果P(x0)为真，则答案小于或等于x0。二分最小值x，把优化问题转化为判定问题P(x)。

- 设所有数值和为M，则二分次数为O(logM)，计算P(x)的时间复杂度为O(n)，总的时间复杂度为O(nlogM).

单调函数求解

- 对于单调函数y=f(x)，给定y值求x值。
- 思路：确定上下界，二分查找。

Thank you!


國立中山大學